Solana PropAMM 套利数据采集与初步分析报告

汇报人：太子瑞

2026 年 4 月 6 日

1 引言

尊敬的导师：

本报告旨在汇报当前 Solana 链上基于 PropAMM (Proprietary Automated Market Maker) 的套利交易采集与分析工作。报告将详细阐述数据分析方法、落库字段结构，总结当前脚本运行遇到的瓶颈及其原因（特别是与以太坊套利检测的对比），并给出初步的结果分析和未来的改进方案。

2 分析方法

本项目的数据采集流水线通过以下几个步骤完成：

1. **扫块过滤**：使用 `getBlock` RPC 接口按指定 slot 范围扫描区块，提取区块内所有的交易签名。
2. **按需拉取**：对比预设的 PropAMM 地址列表，过滤出命中目标程序的交易签名，并将其记录至本地 SQLite 数据库中（`signatures.db`）。
3. **摘要解析与落库**：使用 `getTransaction`（指定 `jsonParsed` 编码格式）获取交易详情。为了节省存储空间并提高查询效率，脚本仅在内存中提取核心的交易流转信息（如代币净余额变动、交易对、是否经过 Jupiter 等聚合器等），将精简后的摘要（Summary）写入 MongoDB 数据库。

3 字段结构说明

存入 MongoDB 的 `tx_summaries` 集合文档结构设计如下：

字段名	数据类型	含义
signature	string	交易签名（Base58），作为逻辑主键。
block_time	int	链上 Unix 时间戳（秒，UTC）。
slot	int	交易所在的 slot。
arbitrage	string[]	**套利路径**：按时间顺序排列的代币转移链路（如 USDC → WSOL → USDC）。
profit	object	**套利利润**：记录交易闭环中净增余额最大的代币地址和增量数额。
propamm_programs	string[]	命中的 PropAMM 相关 program（如配置在 programs.yaml 中的地址）。
via_aggregator	bool	是否在顶层或内部指令中命中聚合器（如 Jupiter）列表。
jupiter_heavy	bool	是否深度依赖 Jupiter（聚合器命中且 Jupiter 指令数 ≥ 3）。
trade_size	object	规模近似：记录单条绝对 ui 变动最大的一腿及其 mint 与变动量。

4 目前存在的问题及原因分析

4.1 采集速度瓶颈

在最近的测试中，使用了多线程并发优化并在剔除噪音程序（如 SoLF）后，分析 100 个区块（包含 1.6 万多条签名）仍需耗时约 1 小时。若以此速度线性外推，Solana 主网一天产生 20 万个区块，光是提取一天的交易数据就需要耗时 2,000 小时，提取一个月（约 600 万个区块）将需要耗时高达 60,000 小时，效率依然无法满足全网高频监测的需要。

4.2 原因对比分析（Solana vs Ethereum）

- **出块速度差异**：以太坊的平均出块时间为 12 秒，而 Solana 约为 400 毫秒。这意味着同等物理时间内的区块数量，Solana 是以太坊的近 30 倍。
- **核心做市商过于庞大**：即便我们已经剔除了明显的机器人噪音，当前系统中类似 pAMM... 和 9H6tua... 这样的高频流动性分发或路由合约极度活跃。仅这 100 个区块中就命中了超过 1.6 万笔相关交易。

4.3 RPC 请求额度不足

当前使用的 Helius RPC 节点提供的额度为 50 万笔交易/月。按照 100 个区块消耗近 1.6 万次请求的比例，50 万次额度仅够支撑大约 3,000 个区块（约合 20 分钟的主网产块量）的深度解析。一旦超出，必须购买更加昂贵的专有商业级节点订阅。

4.4 改进方案

针对上述问题，提出以下三个改进方案：

1. **进一步聚焦核心池子**：继续优化 `programs.yaml`，仅针对特定的小范围流动性池地址进行监听，主动放弃全网级别的 PropAMM 路由中枢以防数据洪流。

2. **企业级 RPC 与 Dedicated 节点**：鉴于 Solana 分析对请求额度的贪婪消耗，如果经费允许，可租用独享节点（Dedicated Node）以避开公有 API 接口的调用次费。

3. **引入 Geyser/RPC 订阅机制**：放弃事后的全盘 `getBlock` 扫块轮询，改用基于 WebSocket 的 `logsSubscribe` 直接在节点端过滤目标事件，仅拉取真正有价值的极少数交易。

5 结果分析

基于目前新的一轮 100 个区块提取解析的 **16,707** 笔交易样本，得出如下初步结论：

- **极高的聚合器依赖度(原结论的重大修正)**:在早期的测试版本中,由于 `aggregators.yaml` 配置文件遗漏了部分核心的 Jupiter 备用路由合约地址(如 `JUP6LkbZbjS1jKKwapdHNy74zcZ3tLUZ` 等)，导致系统误判所有交易均未经过聚合器（**0.0%**）。经过个案溯源（如交易 `uQcSGihf...`）并全网补全 Jupiter 与 Raydium Router 的地址特征后，我们发现：真实的 PropAMM 高频机器人在执行套利时，**深度寄生并依赖 Jupiter 聚合器**的路由系统来寻找全网最佳流动性与最深盘口，而非完全自己构建算法直连底层 DEX。此前的 “0.0% 依赖度” 属于典型的**假阴性（False Negative）**特征提取错误。

- **头部程序占比**：`9H6tua7jkLhdm3w8BvgpTn5L...` 出现 6,960 次（占 **40.6%**），`pAMMBay...` 出现 6,314 次。它们垄断了几乎全部高频交易量。

- **套利链路与利润**：通过新增的 `arbitrage` 链路检测与 `profit` 闭环计算，我们在 1.6 万笔交易中成功提取到了 **4,566** 笔带有明显正向利润的纯套利闭环交易。最常用的中转计价代币为 USDC 和 WSOL。

6 总结

综上所述，我们在 Solana 上的 PropAMM 采集流水线已经跑通并具备了核心数据的提取落库能力，初步验证了这些交易“脱离常规聚合器直连交互”以及“多边原子套利高频爆发”的特征。但由于 Solana 本身的超高吞吐量以及高频机器人的混淆干扰，我们需要通过优化配置剔除噪音以及升级 RPC 基础设施，才能完成大跨度、长周期的数据监测与套利行为的深度挖掘。